Guía de examen

Programación para Ciencia de Datos — Semanas 1 a 4: terminal, entornos virtuales, Git/GitHub, Python I (fundamentos y estructuras) y Python II (funciones y archivos).

Basada en el repositorio del profesor (ESCOMLCD/pcd202701). **No incluye la semana 5** (clases y programación funcional), porque no está en ese repositorio.

Cómo usarla: lee la teoría (secciones 1-7), repasa la tabla de trampas (8) y luego haz los ejercicios (9) **sin mirar las soluciones** (10). En las preguntas de salida, ejecuta el código a mano: una tabla con el valor de cada variable, línea por línea.

- 1. Terminal y entornos virtuales
- 2. Git y GitHub
- 3. Python: tipos, operadores, print y f-strings
- 4. Python: control de flujo
- 5. Python: listas, tuplas, sets y diccionarios
- 6. Python: funciones y scope
- 7. Python: archivos y CSV (patrón de la Práctica 1)
- 8. Trampas típicas de examen
- 9. Ejercicios
- 10. Soluciones

1. Terminal y entornos virtuales

Comandos de navegación y archivos

`pwd`	Muestra en qué carpeta estás.
`ls | ls -l | ls -a | ls -la`	Lista el contenido: normal | detallado | incluye ocultos (empiezan con .) | ambos.
`cd carpeta`	Entra a una carpeta.
`cd .. | cd ../..`	Sube un nivel | sube dos niveles.
`cd ~ | cd /`	Va a tu carpeta personal | va a la raíz del sistema.
`mkdir nombre`	Crea una carpeta.
`mkdir -p a/b/c`	Crea toda la ruta de una vez (-p).
`touch archivo.py`	Crea un archivo vacío.
`cp a.txt b.txt`	Copia un archivo. Con -r copia carpetas completas: cp -r carpeta/ copia/
`mv a.txt carpeta/`	Mueve un archivo. También sirve para renombrar: mv viejo.py nuevo.py
`rm archivo | rm -r carpeta/`	Borra PARA SIEMPRE (no hay papelera). -r para carpetas con contenido.
`cat archivo.txt`	Imprime el contenido del archivo en pantalla.
`clear`	Limpia la pantalla.
`Tab`	Autocompleta nombres de archivos y carpetas.
`code carpeta/`	Abre la carpeta en VS Code.

Ruta absoluta: empieza desde la raíz del sistema (en Windows, C:/Users/...). Solo funciona en esa compu. **Ruta relativa**: se cuenta desde la carpeta donde estás parado (datos/archivo.csv). Python resuelve las rutas relativas desde la carpeta **donde ejecutas el comando**, no desde donde está guardado el .py.

Entornos virtuales (venv) y pip

Un entorno virtual es una instalación aislada de Python para un proyecto: cada proyecto tiene sus propias librerías y versiones, sin chocar con otros proyectos. La carpeta `.venv/` es pesada y específica de cada máquina, por eso **no se sube a Git** (va en .gitignore).

`python --version | git --version`	Verifica que Python y Git estén instalados.
`python -m venv .venv`	Crea el entorno virtual en la carpeta .venv.
`source .venv/Scripts/activate`	Activa el entorno (Git Bash en Windows). En Linux/Mac: source .venv/bin/activate
`deactivate`	Desactiva el entorno.
`pip install paquete`	Instala una librería dentro del entorno activo.
`pip freeze > requirements.txt`	Guarda la lista de librerías instaladas en requirements.txt (vacío si no instalaste nada: es correcto).
`pip install -r requirements.txt`	Instala todas las librerías de ese archivo: así otra persona reproduce tu entorno.

Flujo reproducible: clonar el repo, crear su propio .venv, activarlo, `pip install -r requirements.txt`.

2. Git y GitHub

Conceptos clave

- **Repositorio**: carpeta cuyo historial de cambios controla Git (carpeta oculta .git).
- **Commit**: una "foto" guardada del proyecto, con un mensaje y un código único (hash).
- **Staging**: zona intermedia. `git add` manda archivos ahí; `git commit` guarda lo que está ahí.
- **Rama (branch)**: línea de trabajo paralela. Los cambios en una rama no afectan a main hasta hacer merge.
- **Merge**: trae a la rama actual los cambios de otra rama.
- **Remoto / origin**: el repositorio en GitHub; origin es el nombre por defecto de ese remoto.
- **HEAD**: el commit en el que estás parado. HEAD~1 es un commit antes; HEAD~3, tres antes.
- **Conflicto**: dos ramas cambiaron la misma línea y Git no sabe cuál conservar.

Flujo básico: carpeta de trabajo → `git add` (staging) → `git commit` (historial local) → `git push` (GitHub).

Comandos locales

`git init`	Convierte la carpeta actual en un repositorio.
`git status`	Muestra qué cambió: verde = en staging, rojo = sin agregar / sin seguimiento.
`git add archivo | git add .`	Pasa ese archivo | todos los cambios al staging.
`git commit -m "mensaje"`	Guarda una foto con lo que está en staging.
`git diff`	Muestra las líneas exactas modificadas que aún no están en staging.
`git diff HEAD~1`	Qué cambió respecto al commit anterior.
`git diff abc123 def456`	Compara dos commits por su hash.
`git log`	Historial completo.
`git log --oneline | -5`	Una línea por commit | solo los últimos 5.
`git log --oneline --graph`	Dibuja el historial con ramas y merges.
`git config --global user.name "N"`	Configura tu nombre (y user.email) para firmar commits.

Remotos (GitHub)

`git remote add origin URL`	Conecta tu repo local con uno de GitHub y lo llama origin.
`git remote -v`	Muestra la URL a la que está conectado.
`git clone URL`	Descarga una copia completa de un repo de GitHub.
`git push -u origin main`	Sube main a GitHub; -u la recuerda, después basta git push.
`git push | git pull`	Sube tus commits | trae y fusiona los cambios de otros.
`git fetch`	Trae las referencias del remoto sin fusionar nada.
`git pull origin main --allow-unrelated-histories`	Obliga a unir dos historias que empezaron por separado (repo creado en GitHub y en tu compu por separado).

Ramas e historia

`git branch`	Lista las ramas; la que tiene * es la actual.
`git branch nombre`	Crea una rama sin cambiarte a ella.
`git checkout nombre`	Te cambia a esa rama.
`git checkout -b nombre`	Crea la rama Y te cambia a ella (-b = branch).
`git merge nombre`	Fusiona `nombre` EN la rama actual. Hay que estar parado en la rama que recibe (normalmente main).
`git branch -d nombre`	Borra una rama ya fusionada.
`git branch -a`	Lista ramas locales y remotas.
`git diff main..rama`	Diferencias entre dos ramas (--name-only: solo nombres de archivos).
`git checkout a1b2c3d`	Viaja a ese commit para verlo; se regresa con git checkout main.
`git checkout rama -- archivo`	Trae solo ese archivo desde otra rama, sin cambiarte de rama.
`git checkout HEAD~3 -- archivo`	Recupera el archivo como estaba hace 3 commits.
`git log --oneline --grep="texto"`	Busca commits cuyo mensaje contenga ese texto.
`git log --oneline -- archivo`	Historial de un solo archivo.
`git blame archivo`	Muestra quién modificó cada línea y en qué commit.

Conflictos

Cuando se hace merge y hay conflicto, Git marca el archivo así:

```
<<<<<<< HEAD
tu versión (la rama actual)
=======
la versión de la otra rama
>>>>>>> rama-b
```

Para resolver: 1) abre el archivo y decide qué dejar (o combina ambas); 2) **borra las tres marcas** (`<<<<<<<`, `=======`, `>>>>>>>`); 3) `git add archivo`; 4) `git commit`.

.gitignore y trabajo en pareja

.gitignore lista lo que Git no debe versionar. Para Python: `.venv/`, `__pycache__/`, `*.pyc`, `*.egg-info/` (también archivos con información sensible).

- Para colaborar, el dueño agrega al compañero como colaborador (Settings → Collaborators) y este acepta la invitación; luego ambos hacen `git clone`.
- Antes de hacer `git push`, haz `git pull`. Si el push sale **rejected**, es porque el remoto tiene commits que tú no tienes: haz pull (resuelve conflictos si hay) y vuelve a hacer push.

• Cada quien crea su propio .venv; no se comparte.

3. Python: tipos, operadores, print y f-strings

Tipos y variables

```
edad = 21           # int
precio = 850.50     # float
nombre = "Ana"      # str
activo = True       # bool          type(x) devuelve el tipo
a, b, c = 1, 2, 3   # asignación múltiple
a, b = b, a         # intercambio sin variable temporal
```

Tipado dinámico: una variable puede cambiar de tipo. Nombres: letras, dígitos y guion bajo; no empiezan con dígito, sin espacios, distinguen mayúsculas y no pueden ser palabras reservadas.

Operadores

`+ - *`	Suma, resta, multiplicación. Con str: + concatena y * repite ("5" * 3 es "555").
`/`	División: SIEMPRE devuelve float (17 / 5 = 3.4).
`//`	División entera: solo la parte entera (17 // 5 = 3).
`%`	Módulo: el resto (17 % 5 = 2). n % 2 == 0 significa par.
`**`	Potencia (17 ** 2 = 289).
`== != > < >= <=`	Comparaciones; devuelven True/False. == compara, = asigna.
`and or not`	and: ambas verdaderas; or: al menos una; not: invierte.

Conversiones, print e input

```
int("42")      # 42            int(3.9) = 3, int(-3.9) = -3  (TRUNCA, no redondea)
float("2.50")  # 2.5           str(7)  # "7"
int("hola")    # ValueError    "5" + 3  # TypeError (no se suma str con int)

print("a", "b", sep="-")      # a-b       (sep: separador)
print("x", end=" ")           # sin salto de línea (end: final)
n = input("Edad: ")           # input() SIEMPRE devuelve str -> convertir: int(n)
```

f-strings

```
nombre, prom, ventas = "Carlos", 8.567, 1500000
print(f"{nombre} tiene {prom:.2f}")     # Carlos tiene 8.57   (2 decimales)
print(f"Ventas: ${ventas:,}")           # Ventas: $1,500,000  (miles con coma)
print(f"{'Producto':<12}{'Precio':>8}")  # < alinea izq., > alinea der.
```

Errores y try/except

```
try:
    numero = int("hola")
except ValueError:
    print("No se puede convertir")      # el programa sigue en lugar de romperse
```

4. Python: control de flujo

```
if nota >= 9:        print("Excelente")      # if / elif / else: solo entra UNA rama
elif nota >= 7:      print("Bueno")
else:                print("Reprobado")

for x in [10, 20]:           # recorre elementos
for i in range(5):           # 0,1,2,3,4        (el fin NO se incluye)
for i in range(2, 8):        # 2..7
for i in range(0, 20, 3):    # 0,3,6,...,18     (inicio, fin, paso)
for i, v in enumerate(lista):   # índice Y valor
for letra in "Python":       # recorre caracteres

total = 0
for v in ventas:             # patrón acumulador
    total += v               # total = total + v

while contador > 0:          # repite mientras sea True
    contador -= 1            # sin esto: bucle infinito
```

- **break**: sale del bucle completo. **continue**: salta a la siguiente vuelta.
- Un bucle **while** evalúa la condición ANTES de cada vuelta; el cuerpo corre completo aunque la condición deje de cumplirse a la mitad.

5. Python: listas, tuplas, sets y diccionarios

Listas (mutables, ordenadas)

```
p = ["Laptop", "Mouse", "Teclado", "Monitor", "Impresora"]
p[0]  p[2]  p[-1]  p[-2]        # primero, tercero, último, penúltimo
len(p)                          # 5  (el último índice válido es len(p)-1)
```

Slicing `lista[inicio:fin:paso]`: el **fin no se incluye**.

```
n = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]
n[:3]   -> [0, 10, 20]        n[-3:]  -> [70, 80, 90]      n[2:5] -> [20, 30, 40]
n[::3]  -> [0, 30, 60, 90]    n[::-1] -> al revés          n[3:9:4] -> [30, 70]
n[3:6:-2] -> []   (paso negativo con inicio < fin: lista vacía, sin error)
```

`l.append(x)`	Agrega x al final.
`l.insert(i, x)`	Agrega x en la posición i.
`l.remove(x)`	Elimina la primera aparición del VALOR x.
`l.pop()`	Elimina y DEVUELVE el último (pop(i) el de la posición i).
`l.sort() | l.sort(reverse=True)`	Ordena la lista ORIGINAL (in-place) y devuelve None.
`sorted(l)`	Devuelve una lista NUEVA ordenada; la original no cambia.
`len(l) sum(l) min(l) max(l)`	Cantidad, suma, mínimo, máximo.
`x in l`	True si x está en la lista.

List comprehension: `[expresión for x in iterable if condición]`

```
[x ** 2 for x in range(1, 6)]           # [1, 4, 9, 16, 25]
[x for x in range(1, 21) if x % 2 == 0]  # pares del 1 al 20
[n.upper() for n in ["ana", "luis"]]     # ['ANA', 'LUIS']
```

Tuplas (inmutables)

```
coord = (19.43, -99.13)
lat, lon = coord          # desempaquetado
coord[0] = 20.0           # TypeError: no se pueden modificar
```

Sets (sin duplicados, sin orden)

```
colores = {"rojo", "verde", "rojo"}   # {'rojo', 'verde'}
set([1, 2, 2, 3])                     # elimina duplicados de una lista
A = {1, 2, 3, 4};  B = {3, 4, 5}
A | B  unión {1,2,3,4,5}    A & B  intersección {3,4}
A - B  diferencia {1,2}     A ^ B  simétrica {1,2,5}
"Mouse" in conjunto         # pertenencia (rápido)
```

Diccionarios (llave: valor)

```
est = {"nombre": "Ana", "edad": 21}
est["nombre"]                # 'Ana'   (KeyError si la llave no existe)
est.get("tel", "No hay")     # 'No hay' (no truena)
est["edad"] = 22             # modificar
est["semestre"] = 5          # agregar
est.keys()   est.values()   est.items()
for k, v in est.items(): print(k, v)
{p: len(p) for p in ["sol", "luna"]}   # dict comprehension -> {'sol': 3, 'luna': 4}
```

Patrón de frecuencias (cae seguro):

```
conteo = {}
for v in lista:
    conteo[v] = conteo.get(v, 0) + 1
mas_frecuente = max(conteo, key=conteo.get)
```

Lista de diccionarios (así se ven los datos tabulares antes de pandas): `ventas = [{"producto": "Laptop", "monto": 15000, "vendedor": "Ana"}, ...]` y se recorre con for.

Strings útiles

```
s = "  Hola,Mundo \n"
s.strip()            # "Hola,Mundo"  (quita espacios y \n de los extremos)
s.strip().split(",") # ['Hola', 'Mundo']   (parte en una lista)
" | ".join(["a","b"])  # "a | b"      s.upper()  s.lower()  len(s)
```

6. Python: funciones y scope

```
def sumar(a, b):              # a, b: parámetros
    return a + b             # return DEVUELVE el valor

total = sumar(10, 25)        # 35 (se puede guardar o usar en una expresión)

def con_print(a, b):
    print(a + b)             # solo muestra; devuelve None
r = con_print(3, 4)          # imprime 7;  r vale None

def iva(monto, tasa=0.16):   # valor por defecto
    return monto * (1 + tasa)
iva(1000)                    # usa 0.16
iva(1000, 0.08)              # sobreescribe el default
iva(tasa=0.08, monto=1000)   # argumentos con nombre: el orden no importa

def stats(d):
    return min(d), max(d), sum(d) / len(d)    # varios valores (tupla)
mn, mx, prom = stats([1, 2, 3])               # desempaquetado
```

Scope: una variable creada dentro de una función es **local** y no existe fuera (usarla fuera da NameError). La función puede *leer* una global, pero si le asignas un valor con el mismo nombre, crea una local nueva que oculta a la global; la global no cambia.

```
x = 100
def f():
    x = 999          # local nueva
f()
```

```
print(x)            # 100
```

Docstring: el texto entre triples comillas al inicio de la función; `help(funcion)` lo muestra.

7. Python: archivos y CSV (patrón de la Práctica 1)

```
with open("a.txt", "r", encoding="utf-8") as f:    # with cierra el archivo solo
    todo = f.read()              # TODO el contenido en un solo string
with open("a.txt", "r") as f:
    lineas = f.readlines()       # lista de líneas (cada una con \n al final)
with open("a.txt", "r") as f:
    for linea in f:              # línea por línea (lo mejor para archivos grandes)
        print(linea.strip())     # strip() quita el \n
```

`"r"`	Leer (por defecto). Error si el archivo no existe.
`"w"`	Escribir: SOBREESCRIBE el archivo si ya existe (lo crea si no).
`"a"`	Agregar al final sin borrar lo anterior.

Para escribir: `f.write("texto\n")` (hay que poner el \n tú mismo). Si la carpeta de destino no existe, open(..., "w") da FileNotFoundError.

Leer un CSV sin librerías (el separador puede ser coma o |):

```
with open("datos.csv", "r", encoding="utf-8") as f:
    lineas = f.readlines()
encabezados = lineas[0].strip().split(",")                  # primera línea
datos = [l.strip().split(",") for l in lineas[1:]]          # lista de listas
len(datos), len(encabezados)                                # filas, columnas
idx = encabezados.index("producto")                         # posición por NOMBRE
columna = [fila[idx] for fila in datos]                     # extraer una columna
datos[:5]                                                   # primeras 5 filas
" | ".join(fila)                                            # imprimir una fila
```

Celdas vacías por columna y **estadísticas ignorando lo no numérico** (funciones de la P1):

```
def contar_vacios(filas, encabezados):
    vacios = {col: 0 for col in encabezados}
    for fila in filas:
        for i, valor in enumerate(fila):
            if valor.strip() == "":
                vacios[encabezados[i]] += 1
    return vacios

def estadisticas_columna(valores_str):
    numeros = []
    for v in valores_str:
        try:
            numeros.append(float(v))
        except (ValueError, TypeError):
            continue                # ignora vacíos y texto
    if not numeros:
        return {"n_validos": 0, "minimo": None, "maximo": None}
    return {"n_validos": len(numeros), "minimo": min(numeros), "maximo": max(numeros)}
```

La columna vacía "" no se puede convertir con float(): lanza ValueError y el try/except la salta.

8. Trampas típicas de examen

`17 / 5 vs 17 // 5`	3.4 (float siempre) vs 3 (entera). % es el resto.
`int(3.9) / int(-3.9)`	3 y -3: trunca hacia cero, no redondea.
`"5" * 3 | "5" + 3`	"555" (repite) | TypeError. input() siempre devuelve str.
`range(2, 8)`	2,3,4,5,6,7: el fin no se incluye. Igual en slicing [a:b].
`lista[::-1] | lista[3:6:-2]`	Invierte | lista vacía (paso negativo con inicio < fin).
`r = l.sort()`	r es None; .sort() modifica la original. sorted(l) devuelve una nueva.
`l.remove(x) vs l.pop()`	remove borra por VALOR; pop borra por posición (último) y lo devuelve.
`print en una función sin return`	La función devuelve None.
`x = 999 dentro de una función`	Crea una local; la global no cambia.
`break vs continue`	break termina todo el bucle; continue solo salta esa vuelta.
`print antes/después de c -= 2`	Cambia los valores impresos: traza con tabla.
`d["k"] vs d.get("k", 0)`	KeyError si no existe vs devuelve el default.
`{1, 2, 2, 3}`	Set: los duplicados desaparecen (len 3). Sin orden ni índices.
`tupla[0] = 5`	TypeError: las tuplas son inmutables.
`open(f, "w") dos veces`	La segunda borra lo de la primera. Para agregar: "a".
`linea.split(",") con ",,"`	Genera un elemento "" (celda vacía); no desaparece.
`git add vs git commit vs git push`	Staging | guardar foto local | subir a GitHub.
`git checkout -b x`	Crea la rama x y te cambia a ella.
`git merge x`	Trae x a la rama ACTUAL: primero git checkout main.
`push rejected`	Falta git pull: el remoto tiene commits que tú no.

9. Ejercicios

A. ¿Cuál es la salida?

Haz una tabla con los valores de las variables. Las soluciones están en la sección 10.

A1

```
print(17 // 5, 17 % 5, 17 / 5, 2 ** 5)
```

A2

```
x = "10"
y = 5
print(int(x) + y)
print(x * 2)
```

A3

```
print(int(3.9), int(-3.9), float("2.50"), str(7) + "8")
```

A4

```
n = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]
print(n[2:5])
print(n[-3:])
print(n[::3])
print(n[::-1][:3])
print(n[3:9:4])
print(n[3:6:-2])
```

A5

```
f = ["a", "b", "c"]
f.append("d")
f.insert(1, "z")
f.remove("b")
u = f.pop()
print(f, u, len(f))
```

A6

```
l = [30, 10, 20]
r = l.sort()
print(r, l)
o = [3, 1, 2]
s = sorted(o)
print(o, s)
```

A7

```
for i in range(2, 10, 3):
    print(i, end=" ")
print()
for i, p in enumerate(["x", "y"]):
    print(i, p)
```

A8

```
c = 5
while c > 0:
    c -= 2
    print(c, end=" ")
```

A9

```
for n in [10, -5, 20, -3, 15]:
    if n < 0:
        continue
    if n > 18:
        break
    print(n, end=" ")
```

A10

```
print(len({1, 2, 2, 3}))
A = {1, 2, 3, 4}
B = {3, 4, 5}
print(sorted(A & B), sorted(A - B), sorted(A | B), sorted(A ^ B))
nombre, edad = ("Ana", 25)
print(nombre, edad)
t = (1, 2)
try:
    t[0] = 5
except TypeError:
    print("TypeError")
```

A11

```
d = {"a": 1}
d["b"] = d.get("b", 0) + 5
d["a"] += 1
print(d)
print(d.get("z", "no hay"))
for k, v in d.items():
    print(k, v)
```

A12

```
def con_print(a, b):
    print(a + b)

def con_return(a, b):
    return a + b

r1 = con_print(3, 4)
r2 = con_return(3, 4)
print(r1, r2)

def iva(monto, tasa=0.5):
    return monto * (1 + tasa)

print(iva(100), iva(100, 0.25))
```

A13

```
x = 100
def f():
    x = 999
    return x
y = f()
print(x, y)
try:
    print(z)
except NameError:
    print("NameError")
```

A14

```
print([x * 2 for x in range(1, 8) if x % 2 == 0])
print({p: len(p) for p in ["sol", "luna"]})
print([c.upper() for c in "abc"])
```

A15

```
linea = "Laptop,15000.50,,55-1234\n"
campos = linea.strip().split(",")
print(campos)
print(len(campos), campos[2] == "")
print(" | ".join(campos))
```

A16

```
validos = []
for v in ["12", "abc", "", "7.5"]:
    try:
        validos.append(float(v))
    except ValueError:
        continue
print(validos, len(validos))
```

A17

```
with open("t.txt", "w") as f:
    f.write("A\n")
with open("t.txt", "a") as f:
    f.write("B\n")
with open("t.txt", "w") as f:
    f.write("C\n")
with open("t.txt", "r") as f:
    print(f.readlines())
```

A18

```
conteo = {}
for v in ["a", "b", "a", "", "a", "b"]:
    if v.strip() == "":
        continue
    conteo[v] = conteo.get(v, 0) + 1
print(conteo)
print(max(conteo, key=conteo.get))
```

B. Programa

B1

Escribe una función calcular_promedio(lista) que devuelva el promedio usando solo sum() y len().

B2

Escribe filtrar_mayores(lista, umbral) que devuelva una lista nueva con los números mayores al umbral. Ejemplo: ([10, 25, 5, 30, 15], 20) -> [25, 30].

B3

Escribe resumen_lista(lista) que devuelva un diccionario con n, min, max, promedio y suma.

B4

Escribe es_email_valido(texto): True si tiene exactamente un "@" y al menos un "." después del "@".

B5

Encuentra el máximo y el mínimo de la lista de ventas SIN usar max() ni min(), solo for e if.

ventas = [1200, 850, 1500, 920, 1100, 2000, 750, 1800]

B6

Imprime la tabla de multiplicar de n = 7, del 1 al 10 (formato "7 x 1 = 7").

B7

Convierte total = 3661 segundos a horas, minutos y segundos (pista: // y %).

B8

Cuenta cuántas palabras de la lista tienen más de 5 letras.

palabras = ["gato", "programación", "sol", "computadora", "río", "universidad", "pan"]

B9

Elimina los duplicados de datos y ordena de menor a mayor. Después crea un diccionario palabra -> longitud con una dict comprehension.

datos = [5, 3, 8, 1, 9, 3, 5, 2, 8, 1, 7, 9, 4]

B10

Con el diccionario de calificaciones, calcula el promedio general y la materia con la calificación más alta.

B11

Escribe contar_frecuencias(lista) (ignora vacíos) y mas_frecuente(conteo) que devuelva (valor, veces).

B12

Escribe estadisticas_columna(valores) que reciba strings y devuelva un diccionario con n_validos, minimo y maximo, ignorando vacíos y texto no numérico.

B13

Lee un CSV (con encabezado) sin librerías: imprime filas y columnas, la columna producto y las celdas vacías por columna. (El código de abajo crea primero un archivo de prueba.)

B14

Imprime los números del 1 al 20 que son divisibles entre 3.

B15

Crea numeros.txt con un número por línea (1 al 20), léelo y calcula la suma.

B16

Clasifica las calificaciones (>= 9 Excelente, >= 7 Bueno, >= 6 Suficiente, si no Reprobado) y cuenta cuántas hay de cada tipo.

calificaciones = [9.5, 7.2, 5.8, 8.0, 6.5, 9.0, 4.3]

B17

Con una lista de diccionarios de ventas, calcula cuánto vendió cada vendedor.

C. Git, terminal y entornos virtuales

C1

¿Diferencia entre git add y git commit?

C2

¿Diferencia entre git commit y git push?

C3

¿Qué hace git checkout -b feature/x?

C4

Un git push sale "rejected". ¿Por qué y cómo se arregla?

C5

¿Qué significa git diff HEAD~1?

C6

¿Cómo se ve un conflicto y cómo se resuelve?

C7

¿Qué es .gitignore y qué lleva en un proyecto de Python?

C8

Escribe los comandos para volver repositorio una carpeta, hacer el primer commit y subirlo a un repo vacío de GitHub.

C9

Escribe los comandos para crear una rama, hacer un commit en ella y fusionarla a main.

C10

¿Cómo recuperas hola.py como estaba hace 3 commits?

C11

¿Cómo ves quién modificó cada línea de un archivo y cómo buscas commits por su mensaje?

C12

¿Qué hacen git clone, git pull y git fetch?

C13

¿Para qué sirve un entorno virtual y por qué .venv/ no se sube a Git?

C14

¿Diferencia entre ruta absoluta y relativa? ¿Qué hacen cd ../.. y mkdir -p a/b/c?

C15

Corres python practica1/src/resumen.py pero estás dentro de practica1/src. Si el script usa "datos/archivo.csv", ¿qué pasa?

C16

¿Por qué git merge se ejecuta parado en main y no en la rama de trabajo?

10. Soluciones

A. Salidas

A1

```
3 2 3.4 32
```

// es división entera, % es el resto, / siempre da float y ** es potencia.

A2

```
15
1010
```

int(x) + y suma números (15). x * 2 repite el texto ("1010").

A3

```
3 -3 2.5 78
```

int() trunca hacia cero (no redondea). str(7) + "8" concatena texto.

A4

```
[20, 30, 40]
[70, 80, 90]
[0, 30, 60, 90]
[90, 80, 70]
[30, 70]
[]
```

El fin no se incluye. n[::-1][:3]: primero invierte, luego toma 3. n[3:9:4]: índices 3 y 7. Con paso negativo e inicio menor que el fin, el resultado es lista vacía.

A5

```
['a', 'z', 'c'] d 3
```

Evolución: [a,b,c] -> [a,b,c,d] -> [a,z,b,c,d] -> [a,z,c,d] -> pop quita y devuelve d.

A6

```
None [10, 20, 30]
[3, 1, 2] [1, 2, 3]
```

.sort() modifica la lista y devuelve None; sorted() deja la original igual y devuelve una nueva.

A7

```
2 5 8
0 x
1 y
```

range(2, 10, 3) da 2, 5, 8 (el siguiente, 11, ya pasa el límite). enumerate da índice y valor.

A8

```
3 1 -1
```

Primero resta y luego imprime: 3, 1, -1. Con c = -1 la condición falla, pero ese print ya se ejecutó.

A9

```
10
```

10 se imprime; -5 se salta; 20 activa break y se termina todo el bucle: el 15 nunca se revisa.

A10

```
3
[3, 4] [1, 2] [1, 2, 3, 4, 5] [1, 2, 5]
Ana 25
TypeError
```

El set elimina duplicados (3 elementos). & intersección, - diferencia, | unión, ^ simétrica. Las tuplas no se modifican.

A11

```
{'a': 2, 'b': 5}
no hay
a 2
b 5
```

d.get("b", 0) devuelve 0 porque no existe, y se guarda 5. d["a"] pasa de 1 a 2. get no truena con llaves faltantes.

A12

```
7
None 7
150.0 125.0
```

con_print imprime 7 al llamarla y devuelve None. con_return devuelve 7 sin imprimir. Los argumentos por defecto se usan solo si no pasas ese argumento.

A13

```
100 999
NameError
```

x dentro de f es local: la global sigue valiendo 100. z no existe, por eso NameError.

A14

```
[4, 8, 12]
{'sol': 3, 'luna': 4}
['A', 'B', 'C']
```

Pares del 2 al 6 (2, 4, 6) multiplicados por 2. Un string se recorre letra por letra.

A15

```
['Laptop', '15000.50', '', '55-1234']
4 True
Laptop | 15000.50 |  | 55-1234
```

strip() quita el \n; split(",") con dos comas seguidas deja un elemento vacío (celda vacía) en el medio.

A16

```
[12.0, 7.5] 2
```

float("abc") y float("") lanzan ValueError y se saltan. Los números se guardan como float (12.0).

A17

```
['C\n']
```

"w" crea/sobreescribe, "a" agrega. Después de A, B el último "w" borra todo y deja solo C. readlines() conserva el \n.

A18

```
{'a': 3, 'b': 2}
a
```

Se ignora la celda vacía. max con key=conteo.get devuelve la LLAVE con mayor valor.

B. Programas

B1

```
def calcular_promedio(lista):
    return sum(lista) / len(lista)

print(calcular_promedio([8, 9, 10]))
```

```
9.0
```

B2

```
def filtrar_mayores(lista, umbral):
    return [x for x in lista if x > umbral]

print(filtrar_mayores([10, 25, 5, 30, 15], 20))
```

```
[25, 30]
```

B3

```
def resumen_lista(lista):
    return {"n": len(lista), "min": min(lista), "max": max(lista),
            "promedio": sum(lista) / len(lista), "suma": sum(lista)}

print(resumen_lista([4, 8, 6]))
```

```
{'n': 3, 'min': 4, 'max': 8, 'promedio': 6.0, 'suma': 18}
```

B4

```
def es_email_valido(texto):
    if texto.count("@") != 1:
        return False
    despues = texto.split("@")[1]
    return "." in despues

for e in ["ana@gmail.com", "anagmail.com", "ana@@gmail.com", "ana@gmail"]:
    print(e, es_email_valido(e))
```

```
ana@gmail.com True
anagmail.com False
ana@@gmail.com False
ana@gmail False
```

B5

```
ventas = [1200, 850, 1500, 920, 1100, 2000, 750, 1800]
maximo = ventas[0]
minimo = ventas[0]
for v in ventas:
    if v > maximo:
        maximo = v
    if v < minimo:
        minimo = v
print(maximo, minimo)
```

```
2000 750
```

B6

```
n = 7
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
```

```
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

B7

```
total = 3661
horas = total // 3600
minutos = (total % 3600) // 60
segundos = total % 60
print(f"{horas} hora(s), {minutos} minuto(s), {segundos} segundo(s)")
```

```
1 hora(s), 1 minuto(s), 1 segundo(s)
```

B8

```
palabras = ["gato", "programación", "sol", "computadora", "río", "universidad", "pan"]
cuenta = 0
for p in palabras:
    if len(p) > 5:
        cuenta += 1
print(cuenta)
```

```
3
```

B9

```
datos = [5, 3, 8, 1, 9, 3, 5, 2, 8, 1, 7, 9, 4]
print(sorted(set(datos)))
palabras = ["python", "ciencia", "datos"]
print({p: len(p) for p in palabras})
```

```
[1, 2, 3, 4, 5, 7, 8, 9]
{'python': 6, 'ciencia': 7, 'datos': 5}
```

B10

```
calificaciones = {"Matemáticas": 8.5, "Física": 7.2, "Programación": 9.8,
                  "Inglés": 8.0, "Historia": 7.5}
promedio = sum(calificaciones.values()) / len(calificaciones)
mejor = max(calificaciones, key=calificaciones.get)
print(f"{promedio:.2f}", mejor, calificaciones[mejor])
```

```
8.20 Programación 9.8
```

B11

```
def contar_frecuencias(lista):
    conteo = {}
    for valor in lista:
        if valor.strip() == "":
            continue
        conteo[valor] = conteo.get(valor, 0) + 1
    return conteo

def mas_frecuente(conteo):
    valor = max(conteo, key=conteo.get)
    return valor, conteo[valor]

c = contar_frecuencias(["Laptop", "Mouse", "Laptop", "", "Teclado", "Mouse", "Laptop"])
print(c)
print(mas_frecuente(c))
```

```
{'Laptop': 3, 'Mouse': 2, 'Teclado': 1}
('Laptop', 3)
```

B12

```
def estadisticas_columna(valores):
    numeros = []
    for v in valores:
        try:
            numeros.append(float(v))
        except ValueError:
            continue
    if not numeros:
        return {"n_validos": 0, "minimo": None, "maximo": None}
    return {"n_validos": len(numeros), "minimo": min(numeros), "maximo": max(numeros)}

print(estadisticas_columna(["850.50", "1200", "", "920.30", "abc", "1500"]))
```

```
{'n_validos': 4, 'minimo': 850.5, 'maximo': 1500.0}
```

B13

```
with open("datos.csv", "w", encoding="utf-8") as f:
    f.write("producto,monto,email\nLaptop,15000,ana@gmail.com\nMouse,,luis@hotmail.com\nLaptop,14500,\n")

with open("datos.csv", "r", encoding="utf-8") as f:
    lineas = f.readlines()
encabezados = lineas[0].strip().split(",")
datos = [l.strip().split(",") for l in lineas[1:]]
print(len(datos), len(encabezados))
idx = encabezados.index("producto")
print([fila[idx] for fila in datos])
vacios = {col: 0 for col in encabezados}
for fila in datos:
    for i, valor in enumerate(fila):
        if valor.strip() == "":
            vacios[encabezados[i]] += 1
print(vacios)
```

```
3 3
['Laptop', 'Mouse', 'Laptop']
{'producto': 0, 'monto': 1, 'email': 1}
```

B14

```
for n in range(1, 21):
    if n % 3 == 0:
        print(n, end=" ")
```

```
3 6 9 12 15 18
```

B15

```
with open("numeros.txt", "w") as f:
    for n in range(1, 21):
        f.write(f"{n}\n")

total = 0
with open("numeros.txt", "r") as f:
    for linea in f:
        total += int(linea.strip())
print(total)
```

```
210
```

B16

```
calificaciones = [9.5, 7.2, 5.8, 8.0, 6.5, 9.0, 4.3]
excelentes = buenos = suficientes = reprobados = 0
for c in calificaciones:
    if c >= 9:
        excelentes += 1
    elif c >= 7:
        buenos += 1
    elif c >= 6:
        suficientes += 1
    else:
        reprobados += 1
print(excelentes, buenos, suficientes, reprobados)
```

```
2 2 1 2
```

B17

```
ventas = [
    {"producto": "Laptop", "monto": 15000, "vendedor": "Ana"},
    {"producto": "Mouse", "monto": 350, "vendedor": "Luis"},
    {"producto": "Monitor", "monto": 5000, "vendedor": "Ana"},
    {"producto": "Laptop", "monto": 14500, "vendedor": "Luis"},
]
por_vendedor = {}
for venta in ventas:
    v = venta["vendedor"]
    por_vendedor[v] = por_vendedor.get(v, 0) + venta["monto"]
print(por_vendedor)
```

```
{'Ana': 20000, 'Luis': 14850}
```

C. Git, terminal y venv

C1

add pasa archivos al staging (los marca para el próximo commit); commit guarda en el historial local una foto de lo que está en staging, con un mensaje.

C2

commit guarda la foto solo en tu compu; push sube tus commits al remoto (GitHub) para que otros los vean.

C3

Crea la rama feature/x y te cambia a ella en un solo paso (-b = branch).

C4

El remoto tiene commits que tú no tienes. Haz git pull (resuelve conflictos si aparecen, git add y commit) y vuelve a hacer git push.

C5

Muestra qué cambió entre el commit anterior (HEAD~1) y tu estado actual. HEAD es el commit actual; ~1 es uno atrás.

C6

El archivo trae las marcas <<<<<<< HEAD (tu versión), ======= y >>>>>>> rama (la otra). Se edita dejando el texto final, se borran las tres marcas, se hace git add archivo y git commit.

C7

Lista lo que Git no debe versionar: .venv/, __pycache__/, *.pyc, *.egg-info/ (y archivos con información sensible).

C8

```
git init
git add .
git commit -m "Primer commit"
git branch -m main
git remote add origin https://github.com/USUARIO/repo.git
git push -u origin main
```

Si el repo de GitHub ya tenía un README, antes: git pull origin main --allow-unrelated-histories

C9

```
git checkout -b feature/estadisticas
git add .
git commit -m "Agregar estadisticas"
git checkout main
git merge feature/estadisticas
git branch -d feature/estadisticas
```

C10

git checkout HEAD~3 -- hola.py (o con el hash: git checkout a1b2c3d -- hola.py). Queda en staging.

C11

git blame archivo; y git log --oneline --grep="texto".

C12

clone descarga un repo completo por primera vez; fetch trae las referencias del remoto sin fusionar; pull trae y además fusiona en tu rama.

C13

Aísla las librerías de cada proyecto. .venv/ es pesada y específica de cada máquina; el entorno se reproduce con requirements.txt (pip install -r requirements.txt).

C14

Absoluta: desde la raíz del sistema; relativa: desde la carpeta actual. cd ../.. sube dos niveles. mkdir -p crea a, a/b y a/b/c de una vez.

C15

FileNotFoundError: la ruta relativa se cuenta desde donde ejecutas el comando (practica1/src/datos/archivo.csv no existe). Hay que ejecutar desde la raíz del repo.

C16

Porque merge trae la otra rama a la rama ACTUAL. Para llevar feature/x a main hay que estar en main.